When Should Personal Evidence Become Model Weights?

A Reference-Grounded Method and Novelty Audit for Per-User Offline-to-Online Personalization

Technical Design Note

August 29, 2026

Abstract

This note asks a narrow question: when should evidence about one user remain in an editable prompt or memory, and when should it update that user's model adapter? The question matters because a parameter update is slower, harder to inspect, and more likely to affect unrelated tasks. It is useful only when it improves future behavior beyond what the same evidence can achieve as context.

Prior work already covers per-user adapters, continual adapter updates, retrieval memory, and context-conditioned steering. The closest method, SPRInG, combines a user adapter with retrieved history. EvolveR separately tests whether retrieved experience should remain context or be absorbed into parameters. These papers, rather than the current local implementations, define the evidence boundary of this note.

The remaining hypothesis is a paired placement test. For the same evidence, build one context candidate and one adapter candidate. Compare them on later tasks, then prefer the adapter only if it improves personalization without reducing task success relative to the context candidate. This is a proposed experiment, not a verified result or a settled novelty claim.

1 Decision in Plain Language

A prompt is the default place for new personal information. It is easy to read, change, delete, and limit to one task. A user adapter is worth testing only when a preference repeats across sessions, remains stable, applies to several tasks, and should work without profile text at runtime.

Learning $\Delta\theta_u$ is not useful merely because training is possible. The update must answer a harder question:

> Does the adapter improve later tasks more than giving the same evidence to the model as prompt or memory, without reducing task success?

If the answer is no or remains uncertain, the evidence should stay in editable context.

2 Terms Used in This Note

Post-deployment online update. In this note, an online update occurs after the system has started serving the user. New user evidence arrives, an optimizer changes parameters, and the system produces a new loadable checkpoint. Training on a fixed dataset with fresh model rollouts is often called online RL, but it is not the same as learning from a deployed user's later interactions.

Personalized agent. A personalized agent completes tasks through tools, memory, actions, or clarifying questions while adapting to one user. A system that only changes the style or content of generated text is personalized generation, not necessarily personalized agent execution.

Table 1: Four objects that should not be treated as the same thing.

Object	Meaning	Used when	Main property
Profile prompt	A short text statement of user facts or preferences	Added directly to the input	Easy to inspect and edit, but consumes context
Retrieved memory M_u	Selected records from the user's earlier interactions	Retrieved for a relevant request	Keeps details outside the model, but retrieval can miss or add noise
Training hint h	A correction shown to a teacher during training	Used to make a target or distillation signal	A supervision channel, not necessarily deployed user state
User adapter $\Delta\theta_u$	A learned parameter change for user u	Loaded with the frozen base model	Works without profile text, but is harder to inspect and undo

3 Where Each Design Choice Comes From

The current local code may change or contain errors, so it is not evidence for the method. Table 2 separates published support from choices proposed in this note.

4 Why Not Use a Prompt for Everything?

4.1 Cases where prompt or memory is better

Keep evidence outside the weights when it is:

- observed once or weakly supported;
- temporary, such as a tone request for one report;
- tied to one task, project, or tool;
- likely to change;
- easy to state as a short rule; or
- sensitive enough that the user should be able to inspect and delete it.

This choice has real costs. Retrieval can select the wrong record. Long profiles consume context and are processed again for every request. The model may also ignore a relevant instruction.

4.2 Cases where an adapter may help

Test an adapter when the preference:

- appears in several independent sessions;
- remains stable over time;
- affects several task families;
- cannot be expressed well as one short instruction; and
- should affect behavior even when profile text is absent.

Table 2: Source and status of each design choice.

Design choice	Source	Status here
Chronological split	SPRInG orders each user's records over five periods and uses the earlier 90% of each period for updating and the later 10% for evaluation [3].	Inherited evaluation design
Compare context and adapters	OPPU compares retrieval, profile prompting, and per-user PEFT [1]. SPRInG reports component ablations for adapter and retrieval paths.	Inherited comparison requirement
Test context against parameter absorption	EvolveR keeps retrieved experience loss-masked in its main method and reports a separate unmasked absorption ablation [4].	Motivation; the paired selection rule is proposed
Measure task success	EvolveR uses final-answer correctness as its outcome reward. In this project, completing the task is part of the personalized-agent definition.	Required outcome, not a borrowed algorithm
Safety gate	SPRInG discusses harmful drift, poisoning, and safety filtering only as future work.	Not part of the core method
Confidence intervals	SPRInG reports multi-seed results, p-values, and paired-bootstrap confidence intervals.	Result reporting only, not an update rule
Rollback	The closest papers do not implement a rollback procedure.	Omitted from the core method

An adapter may reduce repeated prompt cost and make the behavior more consistent. It also introduces training cost, forgetting risk, broader behavior changes, and model-management work. These costs make $\Delta\theta_u$ an empirical choice rather than a default.

5 Closest Prior Work

5.1 OPPU and Personalized DPO

OPPU trains one PEFT module from each user's historical behavior [1]. It can also use retrieved history or a generated profile. It is an offline method. It does not decide, for each later observation, whether that observation should remain in memory or enter the adapter.

Personalized RLHF, evaluated through personalized DPO, learns user-conditioned representations and LoRA parameters from offline preference pairs [2]. It does not study chronological post-deployment updates, retrieval memory, or agent task outcomes.

5.2 SPRInG

SPRInG is the closest personalization method [3]. It already contains:

- one evolving LoRA adapter per user;
- likelihood-based selection of incoming interactions;
- a bounded buffer for examples that remain hard after an update;
- relevance-gated retrieval; and
- token-level interpolation between adapter-only and retrieval-conditioned generation.

Thus, "combine memory and an adapter" is not new. SPRInG chooses training examples through likelihood drift, then retains post-update residuals. It does not train matched context and adapter candidates from the same evidence and select between them using later personalization and task success.

SPRInG simulates sequential updates with chronological LongLaMP records. It personalizes text generation. It does not execute tool-based agent tasks or learn from live post-deployment agent outcomes.

5.3 EvolveR

EvolveR studies an agent-level experience loop rather than per-user personalization [4]. It stores distilled principles as retrieved context and updates a global policy with GRPO. Its standard method masks retrieved experience tokens from the training loss.

EvolveR also tests direct experience absorption by unmasking those tokens. Its reported average drops from 0.382 to 0.371. The authors attribute the drop to noisy or irrelevant retrieved principles. This result supports a conservative design: context should not be written into weights without a quality and relevance test.

5.4 CoSteer

CoSteer is a tuning-free alternative [5]. A local small model runs with and without private context. The difference between its two token distributions steers a frozen cloud model. CoSteer does not learn $\Delta\theta_u$. It shows that runtime personalization can be stronger than plain prompting without changing model weights, so it is a useful baseline when privacy and local inference matter.

6 Novelty Audit

Table 3: What the closest methods already cover. "Sequential" means that later records can change model parameters.

Method	Learned object	External user state	Sequential update	Agent task outcomes
OPPU	Per-user PEFT	Optional profile or retrieval	No	No
Personalized DPO	Shared LoRA and user representation	Optional user text	No	No
SPRInG	Per-user LoRA	Residual retrieval buffer	Yes, on a historical stream	No
EvolveR	Global agent policy	Experience Base	Yes, during agent training	Yes, but not per user
CoSteer	None	Private local context	No parameter update	No
Proposed test	Per-user adapter when accepted	Editable memory otherwise	Yes, after deployment	Required

What is not new. Per-user LoRA, personalized DPO, continual adapter updates, retrieval plus parameters, and context-versus-parameter ablations all exist.

Table 4: The narrow mechanism-level difference under study.

Method	How evidence enters weights	How evidence remains external	How an update is accepted
SPRInG	High likelihood-drift examples train the adapter	Hard post-update residuals enter the retrieval buffer	Fixed selection rule; no matched context-versus-adapter validation
EvolveR	Agent trajectories update the policy; experience-token absorption is an ablation	Distilled principles are retrieved as context	Training reward; no per-evidence storage decision
Proposed test	One candidate adapter is trained from eligible evidence	A matched candidate stores the same evidence as context	Later tasks must favor the adapter on personalization without reducing task success

What may remain distinct. The tentative difference is an evidence-level, matched comparison. The same evidence creates a context candidate and an adapter candidate from the same parent checkpoint. Later chronological tasks decide where the evidence should live. The adapter is preferred only when it improves personalization and does not reduce task success relative to context.

Current verdict. This is a defensible research hypothesis, not yet a publication novelty claim. The claim becomes credible only after a broader search confirms that no prior method performs this paired placement and constrained activation procedure.

7 Proposed Paired Placement Test

7.1 State and evidence

For user u at time t, the active state is

$$S_u^{(t)} = \left(\theta_0, \Delta\theta_u^{(t)}, M_u^{(t)}\right), \tag{1}$$

where θ_0 are frozen base parameters, $\Delta\theta_u^{(t)}$ is the user adapter, and $M_u^{(t)}$ is editable memory.

Each evidence record is

$$e_i = (u, t_i, x_i, \tau_i, y_i, f_i, o_i). \tag{2}$$

Here x_i is the task, τ_i is the agent trajectory, y_i is the final answer, f_i is user feedback, and o_i is the environment outcome.

7.2 Matched candidates

Let E_t be a checked batch of new evidence. Build both candidates from the same active state.

Context candidate.

$$M_{u,\text{ctx}} = \text{MemoryUpdate}\left(M_u^{(t)}, E_t\right), \qquad \Delta\theta_{u,\text{ctx}} = \Delta\theta_u^{(t)}. \tag{3}$$

Adapter candidate.

$$\Delta\theta_{u,\mathrm{par}} = \mathrm{TrainAdapter}\left(\Delta\theta_u^{(t)}, E_t\right), \qquad M_{u,\mathrm{par}} = M_u^{(t)}. \tag{4}$$

The adapter candidate uses a fixed, predeclared personalized-training method. It is evaluated without adding the new evidence to its prompt.

7.3 Activation rule

Let V_t contain sessions that occur after E_t but before the untouched final test period. Evaluate the context candidate m_{ctx}, the adapter candidate m_{par}, and the current model m_0 under the same decoding settings.

The adapter candidate dominates the context candidate when

$$m_{\mathrm{par}} \succ m_{\mathrm{ctx}} \quad \Longleftrightarrow \quad \begin{cases} U_{\mathrm{pref}}(m_{\mathrm{par}}; V_t) > U_{\mathrm{pref}}(m_{\mathrm{ctx}}; V_t), \\ U_{\mathrm{task}}(m_{\mathrm{par}}; V_t) \geq U_{\mathrm{task}}(m_{\mathrm{ctx}}; V_t). \end{cases} \tag{5}$$

This is a proposed Pareto rule, not a result inherited from prior work. If the adapter does not dominate, retain the context candidate. If the evidence is contradictory or insufficient, make no placement decision. Confidence intervals may accompany the reported comparison, but they are not part of the selection rule.

Algorithm 1: Reference-grounded paired evidence placement test

Input: Active state $S_u^{(t)}$, new evidence E_t, later validation window V_t
Output: Versioned next state $S_u^{(t+1)}$

Check user identity, time, task, feedback, and task outcome;
if *eligible evidence is insufficient* **then**
    keep the active state;
    return $S_u^{(t)}$;
Build m_{ctx} by storing E_t as editable context;
Build m_{par} by training an adapter on the same E_t;
Evaluate m_0, m_{ctx}, and m_{par} on the same V_t;
if $m_{\mathrm{par}} \succ m_{\mathrm{ctx}}$ *under Eq.* (5) **then**
    select the adapter candidate;
else
    select the context candidate;
return the versioned next state;

8 Experiment That Can Answer the Question

8.1 Chronological data boundary

Split whole sessions before extracting a profile or generating training data:

$$H_u^{\mathrm{past}} \prec E_t \prec V_t \prec H_u^{\mathrm{test}}. \tag{6}$$

The order symbol means "occurs earlier than." The profile, memory, hints, preference pairs, and adapter may use only the records available at that time. The final test period remains untouched until the method is fixed.

8.2 Required arms

Table 5: Minimum comparison needed to identify where personalization comes from.

Arm	Question answered
Frozen base	What happens without personalization?
Base plus profile or memory	How much can explicit context do without training?
User adapter only	Did the weights store useful personal behavior?
User adapter plus memory	Are the two representations complementary?
OPPU-style adapter	Does direct offline per-user training suffice?
SPRInG	Does likelihood-based selective adaptation plus retrieval suffice?
CoSteer, when deployment permits	Can tuning-free local steering avoid a parameter update?
Paired placement test	Does evidence-level placement improve over fixed storage rules?

8.3 Separate outcomes

Do not combine all outcomes into one score. Report:

- preference match on future sessions;
- environment-verified task success;
- prompt tokens, training cost, and inference cost;
- results for each user and task family; and
- uncertainty intervals as evaluation reporting.

Pairwise judges must allow ties and invalid outputs. A style judge cannot replace task correctness. A result from one synthetic user cannot establish a general personalization method.

9 Implementation Order

The shortest credible path is:

1. Select a time-stamped, same-user dataset and define chronological update, validation, and test periods.
2. Implement the published prompt, retrieval, OPPU, and SPRInG baselines from their papers or official repositories.
3. Fix one adapter-training procedure and one context-construction procedure before testing placement.
4. Run base, context-only, adapter-only, and adapter-plus-context arms.
5. Run the paired placement test using the same evidence and validation tasks for both candidates.
6. Report personalization, task success, and cost separately.
7. Extend to tool-using agent trajectories only after the placement result is reproduced.

10 Final Assessment

There is also no general reason to prefer $\Delta\theta_u$ over a prompt. Prompt or memory is better for recent, uncertain, temporary, task-local, changing, or sensitive evidence. An adapter is worth the cost only for repeated and stable behavior that transfers across tasks and works without runtime profile text.

The proposed paired placement test is narrower than "memory plus adapter." It asks both representations to compete using the same evidence and later tasks. SPRInG and EvolveR make this a plausible next experiment, but they also narrow the novelty claim. The paper should claim a new method only after the literature audit and matched experiment both support that claim.

References

[1] Zhaoxuan Tan, Qingkai Zeng, Yijun Tian, Zheyuan Liu, Bing Yin, and Meng Jiang. Democratizing Large Language Models via Personalized Parameter-Efficient Fine-Tuning. In *EMNLP*, 2024. https://aclanthology.org/2024.emnlp-main.372/.

[2] Xinyu Li, Ruiyang Zhou, Zachary C. Lipton, and Leqi Liu. Personalized Language Modeling from Personalized Human Feedback. arXiv:2402.05133, 2024. https://arxiv.org/abs/2402.05133.

[3] Seoyeon Kim and Jaehyung Kim. SPRInG: Continual LLM Personalization via Selective Parametric Adaptation and Retrieval-Interpolated Generation. arXiv:2601.09974, 2026. https://arxiv.org/abs/2601.09974.

[4] Rong Wu, Xiaoman Wang, Jianbiao Mei, and others. EvolveR: Self-Evolving LLM Agents through an Experience-Driven Lifecycle. arXiv:2510.16079, 2025. https://arxiv.org/abs/2510.16079.

[5] Hang Lv, Sheng Liang, Hao Wang, and others. CoSteer: Collaborative Decoding-Time Personalization via Local Delta Steering. arXiv:2507.04756, 2025. https://arxiv.org/abs/2507.04756.